

MyBatis-Plus 从零到原理

项目课程汇总

目录

Spring Boot + MyBatis-Plus 渐进式教学项目	3
第 1 节：数据库与 SQL 基础	5
第 2 节：项目架构与启动	8
第 3 节：实体映射	10
第 4 节：通用 CRUD	12
第 5 节：Lambda 单表查询（重点）	14
第 6 节：Service、事务与数据边界	19
第 7 节：高级特性	21
第 8 节：自定义 XML SQL	23
第 9 节：MyBatis-Plus 简单实现原理	25
第 10 节：代码生成器、测试与排错	28

Spring Boot + MyBatis-Plus 渐进式教学项目

作者: xb 日期: 2026-08-12

这是一个可运行、可调接口、可观察 SQL 的新手教学工程。整堂教学课重点讲解 MyBatis-Plus Lambda 表达式和快速单表查询，并用 demo1 到 demo6 逐步补充 CRUD、分页、高级特性、XML 和代码生成器。

完整教学课件

请从《MyBatis-Plus 从零到原理》[教学课件目录](#) 开始。整堂课分为多个小节，覆盖数据库和 SQL 基础、Spring Boot 分层、实体映射、CRUD、Wrapper、分页、事务、高级插件、自定义 XML、MyBatis-Plus 简单实现原理、代码生成器、测试排错与综合练习；每一节都可跳转到对应代码示例。

Lambda 是本项目的课堂重点，可直接学习 [第 5 节：MyBatis-Plus Lambda 单表查询](#)。

技术版本与环境

- Java 21
- Spring Boot 4.1.0
- MyBatis-Plus 3.5.17 (Spring Boot 4 Starter)
- Maven Wrapper 3.9.16
- MySQL 8

不要使用机器上可能较旧的 mvn，Windows 请运行 mvnw.cmd，macOS/Linux 请运行 ./mvnw。IDEA 的 Project SDK 也要选择 Java 21。

1. 启动项目

默认配置会连接题目指定的 MySQL，并尝试创建 mybatis_plus_demo 数据库、demo_user 表和四条示例数据：

```
$env:JAVA_HOME='C:\Users\xb1\jdk\ms-21.0.11'
.\mvnw.cmd spring-boot:run
```

如账号没有建库权限，请让管理员先执行：

```
CREATE DATABASE mybatis_plus_demo DEFAULT CHARACTER SET utf8mb4;
```

如果暂时不在公司内网，可显式启用内存数据库完成课程练习（默认配置仍是 MySQL）：

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=local"
```

local 数据只存在于当前进程，停止应用后自动清空，适合反复练习。

生产实践中不要把密码写进配置文件。本教学项目支持环境变量覆盖：DB_URL、DB_USERNAME、DB_PASSWORD。

2. 分节练习

启动后打开另一个终端执行：

```
# demo1: 通用列表、按主键查询
curl.exe http://localhost:8080/api/demo1/users
curl.exe http://localhost:8080/api/demo1/users/1
```

```
# demo2: 新增；观察 id、createdAt、updatedAt 自动回填
curl.exe -X POST http://localhost:8080/api/demo2/users -H "Content-Type: application/json" -d
'{"name": "新学员", "age": 22, "email": "student@example.com"}'
```

```
# demo3 (重点): Lambda 单表查询; 条件参数均可选
curl.exe "http://localhost:8080/api/demo3/users/search?keyword=张&minAge=18&maxAge=30&status=1"
curl.exe "http://localhost:8080/api/demo3/users/by-email?email=zhangsan@example.com"
curl.exe "http://localhost:8080/api/demo3/users/by-ids?ids=1,2,3"
curl.exe "http://localhost:8080/api/demo3/users/count?status=1"

# demo4: 分页; 观察 records、total、pages
curl.exe "http://localhost:8080/api/demo4/users/page?current=1&size=2&status=1"

# demo5: 把查询到的 version 原样带回, 成功后 version 会加 1
curl.exe -X PUT http://localhost:8080/api/demo5/users/1 -H "Content-Type: application/json" -d
'{"name": "张三 (已更新)", "age": 21, "email": "zhangsan@example.com", "version": 0}'

# demo5: 逻辑删除和 XML 自定义 SQL
curl.exe -X DELETE http://localhost:8080/api/demo5/users/3
curl.exe "http://localhost:8080/api/demo5/users/active-adults?minAge=18"
```

建议一边调用, 一边看控制台输出的 SQL, 然后依次阅读 `entity` → `mapper` → `service` → `controller`。

3. demo6: 运行代码生成器

先正常启动一次, 确保表已创建; 停止应用后执行:

```
.\mvnw.cmd exec:java "-Dexec.mainClass=generator.com.xb.mybatisplus.MybatisPlusCodeGenerator"
```

生成结果位于 `target/generated-code`, 不会覆盖教学源码。重点阅读 `MybatisPlusCodeGenerator` 中的全局配置、包配置、策略配置和模板引擎配置。

4. 安全测试

```
.\mvnw.cmd clean test
```

测试自动使用内存 H2 数据库, 不连接也不修改远程 MySQL。测试覆盖 `BaseMapper`、XML 查询、分页和逻辑删除。

目录导读

```
src/main/java/com/xb/mybatisplus
├── config      # 分页/乐观锁/防全表操作插件, 字段自动填充
├── controller  # demo1~demo5 REST 教学入口
├── dto         # 参数校验与接口数据边界
├── entity      # 表映射、主键、乐观锁、逻辑删除注解
├── mapper      # BaseMapper 和自定义 SQL 接口
├── service     # IService/ServiceImpl 与事务
└── generator   # demo6 自动代码生成器
```

提醒: 远程数据库属于共享资源时, 不要在课堂中使用真实业务表。本项目所有对象均以 `demo_` 命名。

第 1 节：数据库与 SQL 基础

作者：xb 日期：2026-08-12

[返回课件目录](#)

1. 学习目标

学完本节，你应该能读懂 `demo_user` 的建表语句，手写增删改查 SQL，并理解 SQL 与 Java 对象之间为什么需要“映射”。

2. 从一张表开始

关系型数据库可以先理解为许多有规则的二维表：

id	name	age	email	status
1	张三	20	zhangsan@example.com	1

- 表 (table)：同一类数据的集合，例如用户表。
- 行 (row)：一个具体对象，例如张三。
- 列 (column)：对象的一项属性，例如年龄。
- 主键 (primary key)：唯一标识一行，不能重复。
- 索引 (index)：类似书的目录，提高查找速度，但会增加写入成本和存储。
- 约束 (constraint)：数据库层面的规则，如 `NOT NULL`、`UNIQUE`。

查看项目真实建表脚本：[schema.sql](#)。

```
CREATE TABLE demo_user (  
  id BIGINT NOT NULL AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  PRIMARY KEY (id),  
  UNIQUE KEY uk_demo_user_email (email)  
);
```

这里的 `AUTO_INCREMENT` 表示插入时不传 `id`，MySQL 自动产生下一个 `id`。`UNIQUE` 保证邮箱不重复；这类关键规则应由数据库兜底，不能只依赖 Java 校验。

3. CRUD 是什么

CRUD 是四类数据操作的缩写：Create、Read、Update、Delete。

3.1 新增 INSERT

```
INSERT INTO demo_user(name, age, email, status, created_at, updated_at, version, deleted)  
VALUES ('小白', 18, 'xiaobai@example.com', 1, NOW(), NOW(), 0, 0);
```

3.2 查询 SELECT

SQL 推荐按这个顺序阅读：

```
SELECT id, name, age          -- 要哪些列  
FROM demo_user               -- 从哪张表  
WHERE deleted = 0 AND age >= 18 -- 过滤哪些行
```

```
ORDER BY age DESC      -- 怎样排序
LIMIT 0, 5;            -- 从第 0 条起取 5 条
```

逻辑执行顺序并不完全等于书写顺序。入门阶段先记住：FROM → WHERE → SELECT → ORDER BY → LIMIT。

3.3 更新 UPDATE

```
UPDATE demo_user
SET age = 19, updated_at = NOW()
WHERE id = 1;
```

3.4 删除 DELETE

物理删除会真正移除记录：

```
DELETE FROM demo_user WHERE id = 1;
```

本项目使用逻辑删除，实际执行的是：

```
UPDATE demo_user SET deleted = 1 WHERE id = 1 AND deleted = 0;
```

4. WHERE 是安全边界

缺少 WHERE 的更新会修改整张表：

```
-- 危险：所有用户都会被禁用
UPDATE demo_user SET status = 0;
```

本项目使用 `BlockAttackInnerInterceptor` 阻止无条件全表更新/删除。查看[防全表操作配置](#)。数据库权限、备份和代码审查仍然不可省略。

5. SQL 参数为什么不能直接拼字符串

错误思路：

```
"SELECT * FROM demo_user WHERE name = '" + userInput + "'"
```

如果输入包含恶意 SQL，可能产生 SQL 注入。MyBatis 的 `# {name}` 使用预编译参数，SQL 结构和数据分开传给数据库：

```
SELECT * FROM demo_user WHERE name = ?
```

项目 XML 的安全参数示例：[UserMapper.xml](#)。除非完全理解风险，不要使用 `${}` 接收外部输入。

6. 表和 Java 对象的差异

数据库列常用下划线：`created_at`；Java 属性常用驼峰：`createdAt`。MyBatis 的映射工作，就是把查询结果的一行转换为一个 `User` 对象。本项目开启了下划线转驼峰：[application.yml](#)。

数据库	Java
BIGINT	Long
INT	Integer
VARCHAR	String
DATETIME	LocalDateTime
created_at	createdAt

7. 动手实验

练习 SQL 位于 [lesson-queries.sql](#)。先猜结果，再逐条执行：

1. 查询所有未删除用户。
2. 查询年龄在 18 ~ 30 岁之间的启用用户。
3. 按年龄倒序取前两名。
4. 修改一个用户并观察 `updated_at`。
5. 把物理删除改写为逻辑删除。

8. 本节检查

- `WHERE` 和 `HAVING` 的过滤对象有什么不同？（本项目暂未使用分组，先知道 `HAVING` 用于分组后的结果。）
- 为什么邮箱既做接口校验，又做数据库唯一约束？
- `LIMIT 5, 10` 中两个数字分别是什么？

[返回课件目录](#) · [下一节：项目架构与启动](#)

第 2 节：项目架构与启动

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. 技术栈分别负责什么

- Spring Boot：启动应用、创建和管理对象、读取配置、提供 Web 服务。
- MyBatis：执行 SQL、绑定参数、把结果映射为 Java 对象。
- MyBatis-Plus：在 MyBatis 上提供通用 CRUD、条件构造器、分页和代码生成等能力。
- JDBC Driver：按 JDBC 标准与 MySQL 通信。
- Maven：解析依赖、编译、测试、打包。

依赖声明可查看 [pom.xml](#)。项目使用 Spring Boot 4 对应的 `mybatis-plus-spring-boot4-starter`，不要同时再引入普通 MyBatis Starter，以免自动配置冲突。

2. 一次请求经过哪些层

以 `GET /api/demo1/users` 为例：

```
浏览器/curl
→ Spring MVC 找到 UserLessonController.list()
→ userService.list()
→ UserMapper 动态代理
→ BaseMapper 中预先注册的 selectList SQL
→ MyBatis 参数处理和插件链
→ JDBC Driver
→ MySQL
→ ResultSet 映射成 List<User>
→ Jackson 序列化为 JSON
```

跳转查看：

- [Controller 的 list 方法](#)
- [UserService 继承 IService](#)
- [UserMapper 继承 BaseMapper](#)

3. 启动项目

3.1 检查 Java 和 Maven Wrapper

```
$env:JAVA_HOME='C:\Users\xb1\.jdk\ms-21.0.11'
.\mvnw.cmd -v
```

输出应显示 Java 21 和 Maven 3.9.16。

3.2 无公司内网：使用 local 模式

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=local"
```

它使用 H2 内存数据库，配置见 [application-local.yml](#)。停止应用后数据消失，适合练习。

3.3 有公司内网：使用 MySQL

```
.\mvnw.cmd spring-boot:run
```

默认连接配置见 `application.yml`。实际项目应通过 `DB_URL`、`DB_USERNAME`、`DB_PASSWORD` 环境变量提供凭据，不应把生产密码提交到版本库。

3.4 发出第一个请求

```
curl.exe http://localhost:8080/api/demo1/users
```

同时观察应用控制台中的 `Preparing` (SQL) 与 `Parameters` (参数)。这是学习 ORM 最有效的方法之一：始终确认框架最终发出了什么 SQL。

4. Spring Boot 自动配置做了什么

启动类上的 `@SpringBootApplication` 组合了配置、自动配置和组件扫描。查看[启动类](#)。

启动时大致发生：

1. 读取 `application.yml` 并创建 `DataSource`。
2. MyBatis-Plus Starter 创建 `SqlSessionFactory` 和 `SqlSessionTemplate`。
3. `@MapperScan` 找到 Mapper 接口，为它们注册代理 Bean。
4. 组件扫描找到 Controller、Service、Config、异常处理器。
5. 执行初始化 SQL，随后启动内嵌 Web Server。

Spring 的依赖注入让 Controller 只声明构造参数，不需要自己 `new UserServiceImpl()`。查看[构造器注入](#)。

5. 分层不是为了多建文件

层	本项目类	核心职责	不应承担
Controller	<code>UserLessonController</code>	HTTP 参数、状态码、响应	拼复杂 SQL、开启业务事务
DTO	<code>CreateUserRequest</code>	外部输入结构和校验	数据库持久化行为
Service	<code>UserServiceImpl</code>	业务规则、事务、编排	理解 HTTP 细节
Mapper	<code>UserMapper</code>	数据访问	业务流程
Entity	<code>User</code>	数据库行映射	接收任意外部字段

小项目可以精简层次，但每种职责依然要能说清楚。

6. 配置优先级入门

`${DB_USERNAME:sip}` 表示：优先使用环境变量 `DB_USERNAME`，没有才使用默认值 `sip`。Profile 配置会覆盖基础配置，所以 `local` 能把 MySQL `DataSource` 换成 H2。

7. 动手实验

1. 把 `server.port` 临时改成 9090，验证请求地址也要变化。
2. 注释掉 `@MapperScan`，观察启动或调用时的错误，再恢复。
3. 找到启动日志中的连接池、Mapper 和 Tomcat 初始化信息。

[返回课件目录](#) · [下一节：实体映射](#)

第 3 节：实体映射

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. Entity 是数据库行在 Java 中的表示

核心示例是 `User.java`。一条查询结果会被映射成一个 `User`，多行会成为 `List<User>`。

```
@TableName("demo_user")
public class User {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String name;
}
```

2. 常用映射注解

注解	项目示例	作用
<code>@TableName</code>	User 第 24 行	类对应哪张表
<code>@TableId</code>	User 第 28 行	标记主键并指定生成策略
<code>@TableField</code>	自动填充字段	字段配置、自动填充或排除非表字段
<code>@Version</code>	version 字段	标记乐观锁版本号
<code>@TableLogic</code>	deleted 字段	标记逻辑删除列

字段名不符合驼峰规则时可显式指定：`@TableField("user_name")`。非数据库字段使用 `@TableField(exist = false)`。

3. 主键策略

本项目使用 `IdType.AUTO`，依赖 MySQL 自增。常见选择：

- `AUTO`：数据库自增，直观，适合单库入门。
- `ASSIGN_ID`：MyBatis-Plus 生成分布式 ID，常见实现是雪花算法。
- `INPUT`：必须由调用方赋值。

主键策略必须与表结构一致。例如实体写 `AUTO`，数据库却没有自增，插入就可能失败。

4. Java 类型选择

实体字段优先使用包装类型 `Integer/Long`，因为它们能表示 `null`。更新时 `null` 常表示“不修改此字段”；基本类型 `int` 的默认值 `0` 无法区分“未传”和“明确传 `0`”。

时间字段使用 `LocalDateTime`，而不是旧的 `Date`。数据库、JDBC URL 和应用时区要一致，项目 JDBC URL 指定了 `serverTimezone=Asia/Shanghai`。

5. Entity 和 DTO 为什么分开

创建接口只允许客户端提供姓名、年龄和邮箱：`CreateUserRequest`。如果直接接收 `User`，客户端可能提交 `deleted=1`、伪造 `version` 或修改内部状态。

请求 JSON → DTO（外部边界）→ Service 转换 → Entity（数据库边界）

DTO 上的 `@NotBlank`、`@Email` 属于快速反馈；数据库的非空和唯一约束属于最终数据完整性保障，两者互补。

6. 自动映射规则

`map-underscore-to-camel-case: true` 会完成：

```
created_at → createdAt
updated_at → updatedAt
```

如果列名和属性差异更大，就需要 `@TableField` 或 XML `resultMap`。

7. 动手实验

1. 在表中设计一个 `nickname` 可空列，并在实体添加对应属性。
2. 设计一个只用于接口显示、数据库不存在的 `displayName` 字段，使用 `exist = false`。
3. 思考金额为什么不应使用 `double`，而应选择数据库 `DECIMAL` 与 Java `BigDecimal`。

[返回课件目录](#) · [下一节：通用 CRUD](#)

第 4 节：通用 CRUD

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. 最小 Mapper

`UserMapper` 只需继承：

```
public interface UserMapper extends BaseMapper<User> {  
}
```

它立即拥有 `insert`、`selectById`、`selectList`、`updateById`、`deleteById` 等方法。泛型 `User` 告诉框架操作哪种实体、哪张表。

2. 查询

Controller 中的列表和主键查询：

```
userService.list();  
userService.getById(id);
```

底层大致对应：

```
SELECT id, name, age, email, status, created_at, updated_at, version, deleted  
FROM demo_user  
WHERE deleted = 0;
```

```
SELECT ... FROM demo_user WHERE id = ? AND deleted = 0;
```

底层 `getById` 在记录不存在时会返回 `null`。本项目通过 `getRequiredById` 把这种情况转换为业务异常，再由全局异常处理器返回 HTTP 404，避免出现含义模糊的 `200 + null`。

3. 新增

查看完整实现：`UserServiceImpl.create`。

步骤是 DTO → Entity → `save(user)`。执行成功后，自增 id 和自动填充时间会回写到对象。

```
curl.exe -X POST http://localhost:8080/api/demo2/users `   
-H "Content-Type: application/json" `   
-d '{"name":"新学员","age":22,"email":"student@example.com"}'
```

4. 更新

查看乐观锁更新实现。`updateById` 只会用主键定位记录；默认字段策略通常不会把 `null` 更新进数据库。

初学者常犯的错误是先构造一个缺少 id 的实体再调用 `updateById`。没有主键，框架不知道更新哪一行。

5. 删除

查看删除接口。虽然代码调用 `removeById`，由于实体使用 `@TableLogic`，最终是逻辑删除的 UPDATE。

6. Mapper 和 Service 两套 API 如何选

- Mapper：贴近 SQL，一次数据库操作；返回受影响行数或数据。

- Service：在 Mapper 上提供批量操作、事务编排和更接近业务的复用点。

Controller 优先依赖自己的业务 Service，避免以后业务规则散落在各接口。简单教学查询仍可展示 Mapper 的直接调用。

7. 返回值必须检查

`insert/update/delete` 返回的受影响行数，以及 Service 的布尔值，都不应盲目忽略。0 可能表示记录不存在、条件不匹配或乐观锁冲突。本项目更新失败时抛出明确异常：[冲突处理](#)。

8. 动手实验

1. 调用新增接口，再用返回的 id 查询。
2. 删除该 id，再次查询，观察逻辑删除效果和 HTTP 404。
3. 第二次删除同一个 id，观察“写操作影响 0 行”如何转换成 404。
4. 尝试插入重复邮箱，阅读数据库异常中的唯一索引名称。

[返回课件目录](#) · [下一节：Lambda 单表查询](#)

第 5 节：Lambda 单表查询（重点）

作者：xb 日期：2026-08-12

[上一节](#) · [返回课件目录](#) · [查看完整代码](#)

本节是整堂课的重点，把Lambda 单表查询最常用的写法集中在一张课件中，方便课堂演示和课后复习。

本节示例导航

下面每个示例都可以直接运行，并能跳转到具体代码行：

业务场景	HTTP 接口	Lambda 查询代码	自动测试
根据唯一邮箱查一条	Controller 第 94 行	findOneByEmail 第 37 行	测试第 91 行
姓名、年龄、状态组合查询	Controller 第 85 行	search 第 49 行	测试第 96 行
根据多个 id 查询	Controller 第 100 行	findByIds 第 67 行	测试第 101 行
统计启用用户数量	Controller 第 106 行	countByStatus 第 80 行	测试第 105 行
按状态分页	Controller 第 112 行	page 第 86 行	测试第 108 行

本节返回字段来自 User 实体第 24 行，练习数据来自 local-data.sql。

一、固定使用步骤

```
// 1. 创建 User 实体对应的 Lambda 查询条件构造器
LambdaQueryWrapper<User> wrapper = Wrappers.<User>lambdaQuery()
// 2. 添加查询条件
    .eq(User::getStatus, 1)
// 3. 添加排序条件
    .orderByAsc(User::getAge);

// 4. 根据需要选择 BaseMapper 查询方法
List<User> users = userMapper.selectList(wrapper);
```

记忆口诀：选方法、建 Wrapper、加条件、执行查询。

二、快速选择查询方法

查询目标	推荐方法	示例
已知主键，查询一条	<code>selectById(id)</code>	主键查询示例
按唯一条件查询一条	<code>selectOne(wrapper)</code>	邮箱精确查询
按条件查询多条	<code>selectList(wrapper)</code>	姓名、年龄、状态组合查询
只统计数量	<code>selectCount(wrapper)</code>	统计启用用户数量
分页查询	<code>selectPage(page, wrapper)</code>	查询指定页

三、查一条：eq + selectOne

场景：登录或资料查询时，根据唯一邮箱 `zhangsan@example.com` 查找张三。

```
User user = userMapper.selectOne(
    Wrappers.<User>lambdaQuery()
```

```
// eq 表示“等于”，大致生成 email = ?
.eq(User::getEmail, email)

);
```

- 适用于主键、唯一邮箱等最多返回一条数据的条件。
- 如果实际返回多条，`selectOne` 会抛出异常。
- 接口调用：GET /api/demo3/users/by-email?email=zhangsan@example.com
- 预期结果：返回张三这一条用户数据。
- 跳转代码：接口入口 → Lambda 查询实现 → 测试断言。

四、查多条：可选条件 + selectList

场景：查询姓名包含“张”、年龄在 18~30 岁之间、状态为启用的用户。

```
LambdaQueryWrapper<User> wrapper = Wrappers.<User>lambdaQuery()
    // boolean 为 false 时，不会把该条件加入 SQL
    .like(keyword != null && !keyword.isBlank(), User::getName, keyword)
    .ge(minAge != null, User::getAge, minAge)
    .le(maxAge != null, User::getAge, maxAge)
    .eq(status != null, User::getStatus, status)
    .orderByAsc(User::getAge)
    .orderByAsc(User::getId);

List<User> users = userMapper.selectList(wrapper);
```

大致生成：

```
SELECT ... FROM demo_user
WHERE deleted = 0
  AND name LIKE ?
  AND age >= ?
  AND age <= ?
  AND status = ?
ORDER BY age ASC, id ASC;
```

实际请求：

```
GET /api/demo3/users/search?keyword=张&minAge=18&maxAge=30&status=1
```

使用项目初始化数据时，预期只返回张三。删掉 `maxAge` 参数后，`.le(false, ...)` 不会进入 SQL，这正是可选条件写法的作用。

跳转代码：接口参数接收 → like、ge、le、eq 实现 → 组合查询测试。

五、按集合查询：in + selectList

场景：用户勾选了 id 为 1、2、3 的三条记录，需要一次查询这些用户。

```
if (ids == null || ids.isEmpty()) {
    return List.of();
}

return userMapper.selectList(
    Wrappers.<User>lambdaQuery()
        .in(User::getId, ids)
        .orderByAsc(User::getId)
);
```

`in` 大致生成 `id IN (?, ?, ?)`。空集合要提前处理，防止条件被跳过后意外查询整张表。

- 接口调用：GET /api/demo3/users/by-ids?ids=1,2,3
- 预期结果：返回 id 位于 1、2、3 集合中的用户，并按 id 升序排列。

- 跳转代码：[接口入口](#) → [空集合保护和 in 查询](#) → [正常集合与空集合测试](#)。

六、统计数量：selectCount

场景：页面上只显示“启用用户共多少人”，不需要读取每个用户的完整资料。

```
Long count = userMapper.selectCount(
    Wrappers.<User>lambdaQuery()
        .eq(User::getStatus, 1)
);
```

只需要数量时不要先 `selectList` 再调用 `size()`，应让数据库直接执行 `COUNT`。

- 接口调用：`GET /api/demo3/users/count?status=1`
- 返回示例：`3`，表示初始化数据中有 3 个启用且未逻辑删除的用户。
- 跳转代码：[接口入口](#) → [selectCount 实现](#) → [数量断言](#)。

七、分页查询：selectPage

场景：用户列表每页只展示 2 条，并且只查看启用用户。

```
Page<User> page = userMapper.selectPage(
    Page.of(current, size),
    Wrappers.<User>lambdaQuery()
        .eq(status != null, User::getStatus, status)
        .orderByDesc(User::getCreatedAt)
);
```

常用分页结果：

属性	含义
<code>page.getRecords()</code>	当前页数据
<code>page.getTotal()</code>	符合条件的总记录数
<code>page.getCurrent()</code>	当前页码
<code>page.getSize()</code>	每页数量
<code>page.getPages()</code>	总页数

实际请求：

```
GET /api/demo4/users/page?current=1&size=2&status=1
```

返回结构示例：

```
{
  "records": [
    {"id": 3, "name": "王小明", "age": 16, "status": 1},
    {"id": 2, "name": "李四", "age": 28, "status": 1}
  ],
  "total": 3,
  "size": 2,
  "current": 1,
  "pages": 2
}
```

跳转代码：[分页参数入口](#) → [Lambda 条件分页实现](#) → [records 和 total 测试](#) → [分页插件配置](#)。

八、常用 Lambda 条件对照

MyBatis-Plus 写法	含义	大致 SQL
<code>.eq(User::getStatus, 1)</code>	等于	<code>status = ?</code>
<code>.ne(User::getStatus, 0)</code>	不等于	<code>status <> ?</code>
<code>.gt(User::getAge, 18)</code>	大于	<code>age > ?</code>
<code>.ge(User::getAge, 18)</code>	大于等于	<code>age >= ?</code>
<code>.lt(User::getAge, 60)</code>	小于	<code>age < ?</code>
<code>.le(User::getAge, 60)</code>	小于等于	<code>age <= ?</code>
<code>.between(User::getAge, 18, 30)</code>	范围	<code>age BETWEEN ? AND ?</code>
<code>.like(User::getName, "张")</code>	模糊包含	<code>name LIKE '%张%'</code>
<code>.likeLeft(User::getName, "三")</code>	左侧模糊	<code>name LIKE '%三'</code>
<code>.likeRight(User::getName, "张")</code>	右侧模糊	<code>name LIKE '张%'</code>
<code>.in(User::getId, ids)</code>	属于集合	<code>id IN (?, ?)</code>
<code>.isNull(User::getEmail)</code>	字段为空	<code>email IS NULL</code>
<code>.isNotNull(User::getEmail)</code>	字段不为空	<code>email IS NOT NULL</code>
<code>.orderByAsc(User::getAge)</code>	升序	<code>ORDER BY age ASC</code>
<code>.orderByDesc(User::getId)</code>	降序	<code>ORDER BY id DESC</code>

九、AND、OR 和括号

```
// status = 1 AND (age < 18 OR age > 60)
wrapper.eq(User::getStatus, 1)
    .and(w -> w.lt(User::getAge, 18)
        .or()
        .gt(User::getAge, 60));
```

内层 `w -> ...` 用于生成括号。复杂查询不要只看 Java 链式代码，还要查看控制台打印的实际 SQL。

十、为什么使用 `User::getName`

```
// 不推荐：列名是字符串，字段重命名时不容易发现问题
wrapper.eq("name", keyword);

// 推荐：方法引用可以参与编译检查和 IDE 重构
wrapper.eq(User::getName, keyword);
```

MyBatis-Plus 的简单处理过程：

```
User::getName
→ 解析为 User 的 name 属性
→ 根据实体映射找到 name 列
→ Wrapper 保存条件和值
→ BaseMapper 生成并执行参数化 SQL
```

十一、接口练习

```
# 多条件查询
curl.exe "http://localhost:8080/api/demo3/users/search?keyword=张&minAge=18&maxAge=30&status=1"

# 根据唯一邮箱查一条
curl.exe "http://localhost:8080/api/demo3/users/by-email?email=zhangsan@example.com"

# 根据多个 id 查询
curl.exe "http://localhost:8080/api/demo3/users/by-ids?ids=1,2,3"

# 统计启用用户数量
curl.exe "http://localhost:8080/api/demo3/users/count?status=1"

# 带状态条件的分页查询
curl.exe "http://localhost:8080/api/demo4/users/page?current=1&size=2&status=1"
```

十二、新手避坑

- `selectOne` 只用于最多返回一条的条件。
- `selectList(null)` 可能查询整张表，使用前要确认数据规模。
- `in` 的集合为空时应提前处理。
- 可选参数优先使用条件方法的第一个 `boolean` 参数。
- 排序字段不要直接接收前端字符串并拼接到 SQL。
- `Lambda Wrapper` 负责构造条件，但不能代替数据库索引设计。
- 写完组合条件后，查看 SQL 并运行[Lambda 单表查询集成测试](#)。

[返回课件目录](#) · [下一节：Service、事务与数据边界](#)

第 6 节：Service、事务与数据边界

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. Service 表达业务动作

`UserService` 不只是重复 Mapper 方法，还定义了 `create` 和 `updateWithOptimisticLock` 这类业务语义。

`ServiceImpl<UserMapper, User>` 提供通用实现，而本项目的 Service 负责：

- DTO 转 Entity；
- 设置业务默认值 `status=1`；
- 检查更新是否成功；
- 控制事务边界。

2. 什么是事务

事务把多条数据库操作视为一个整体，满足“全部成功或全部失败”。经典转账：

```
账户 A -100  
账户 B +100
```

如果第一条成功、第二条失败而没有事务，钱就凭空消失。`@Transactional` 会在方法开始前取得连接并开启事务；正常返回时提交，符合回滚规则的异常抛出时回滚。

查看项目事务位置：`create`、`update`。

3. 常见事务误区

- `private` 方法或同类内部 `this.method()` 调用，通常绕过 Spring 代理，事务可能不生效。
- 捕获异常后不再抛出，Spring 会认为方法正常完成并提交。
- 默认主要对运行时异常回滚；需要根据业务明确 `rollbackFor`。
- 跨数据库、消息队列的事务不是一个简单注解就能解决。
- 事务时间应尽量短，不要在事务中做长时间网络调用。

4. DTO、Entity、VO

```
DTO：客户端允许输入什么  
Entity：数据库保存什么  
VO：接口允许输出什么
```

本项目为保持入门代码紧凑，响应暂时直接返回 Entity；生产项目通常增加 VO，以避免泄露内部字段 `deleted`、`version`，也避免数据库结构变化直接破坏 API。

参数校验入口：`CreateUserRequest` 和 Controller 的 `@Valid`。校验失败由全局异常处理器转换成 JSON。

5. HTTP 状态码也是接口设计

- `200 OK`：查询或更新成功。

- 201 Created: 创建成功, 本项目新增接口使用它。
- 204 No Content: 删除成功且无响应体。
- 400 Bad Request: 参数格式或校验失败。
- 404 Not Found: 资源不存在。
- 409 Conflict: 乐观锁等状态冲突。

6. 动手实验

1. 新建 `UserResponse`, 只输出 `id`、`name`、`age`、`email` 和时间。
2. 用一个私有转换方法把 `User` 转为 `UserResponse`。
3. 设计 “批量创建两个用户, 第二个邮箱重复” 的实验, 验证事务是否回滚第一个插入。

[返回课件目录](#) · [下一节: 高级特性](#)

第 7 节：高级特性

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. 自动填充

实体使用 `@TableField(fill = ...)` 声明填充时机：[User 时间字段](#)。真正提供值的是 [AuditMetaObjectHandler](#)。

```
执行 insert
→ MyBatis-Plus 发现 INSERT 填充字段
→ MetaObjectHandler 写入 createdAt/updatedAt
→ 生成并执行 SQL
```

`strictInsertFill` 通常只在当前值为空时填充，避免覆盖业务明确设置的值。

2. 乐观锁

乐观锁适合“冲突不常发生”的场景。两个操作者都读到 `version=0`：

```
A 更新: WHERE id=1 AND version=0 → 成功, version 变 1
B 更新: WHERE id=1 AND version=0 → 影响 0 行, 提示重新读取
```

实体标记见 `@Version`，插件见[乐观锁配置](#)，业务检查见[更新结果判断](#)。

乐观锁不会自动重试业务操作。盲目重试可能覆盖别人的修改，通常应提示用户查看最新数据后决定。

3. 逻辑删除

实体标记见 `@TableLogic`，全局值配置见 [application.yml](#)。

通用查询自动增加 `deleted=0`，通用删除改写成 `deleted=1`。但手写 XML SQL 必须自己处理逻辑删除条件，本项目已在[自定义 SQL](#)显式添加。

逻辑删除的代价：

- 数据仍占空间；
- 唯一索引设计更复杂；
- 所有手写 SQL 都必须遵守过滤规则；
- 它不是审计系统，也不能替代备份。

4. 防全表更新与删除

[BlockAttackInnerInterceptor](#) 会阻止明显缺少条件的全表更新/删除。它是一道护栏，不代表任何带 `WHERE` 的 SQL 都安全，例如 `WHERE id > 0` 仍可能覆盖全表。

5. 插件顺序

MyBatis-Plus 插件会依次改写 SQL。项目顺序是：

```
乐观锁 → 防全表操作 → 分页
```

分页一般靠后，让其他插件先完成 SQL 处理。新增租户、数据权限等插件时，要阅读官方说明并用最终 SQL 验证顺序。

6. 动手实验

1. 连续两次用相同 version 更新同一用户，第二次应返回 409。
2. 逻辑删除后直接在数据库查看该行，再通过普通接口查询。
3. 临时写一个无条件 update Wrapper，在隔离数据库验证防攻击插件，再删除实验代码。

[返回课件目录](#) · [下一节：自定义 XML SQL](#)

第 8 节：自定义 XML SQL

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. 什么时候离开通用 CRUD

BaseMapper 适合单表常见操作。遇到复杂关联、聚合报表、数据库特有函数或需要精确控制 SQL 时，应该编写清晰的自定义 SQL，而不是强行堆叠 Wrapper。

项目接口声明：UserMapper.findActiveAdults。XML 实现：UserMapper.xml。

2. namespace 与 id 如何绑定

```
<mapper namespace="com.xb.mybatisplus.mapper.UserMapper">
  <select id="findActiveAdults">...</select>
</mapper>
```

namespace + id 必须等于 Mapper 接口全限定名 + 方法名。参数 @Param("minAge") 对应 XML 的 #{minAge}。

3. #{ } 与 \${ }

- #{value}：变成 JDBC ? 参数，默认安全，应优先使用。
- \${value}：直接文本替换，可能 SQL 注入，只适合经过严格白名单控制的 SQL 结构片段。

排序字段不能直接接受 \${sort}。正确方式是在 Java 中把有限的外部选项映射为固定列，或者在 XML 用 <choose> 白名单。

4. resultType 与 resultMap

本项目列名可按驼峰自动映射，所以使用 resultType="...User"。复杂映射可使用 resultMap：

```
<resultMap id="userMap" type="com.xb...User">
  <id column="id" property="id"/>
  <result column="created_at" property="createdAt"/>
</resultMap>
```

联表的一对一、一对多也可用 resultMap，但要警惕 N+1 查询：查询一批用户后又为每个用户单独查询一次明细，会让 SQL 数随数据量增长。

5. 动态 SQL

MyBatis XML 支持 <if>、<where>、<foreach>、<choose>：

```
<where>
  deleted = 0
  <if test="minAge != null">AND age >= #{minAge}</if>
</where>
```

XML 中 < 是特殊字符，所以小于号通常写成 <，或使用 CDATA。

6. XML 常见错误定位

- Invalid bound statement：namespace、id、XML 扫描路径不匹配。

- `Parameter 'x' not found: @Param` 名和 XML 引用不一致。
- 字段全是 null：列名与属性映射不一致。
- 逻辑删除数据被查出：手写 SQL 忘记 `deleted=0`。

XML 扫描位置配置见 [application.yml](#)。

7. 动手实验

新增 `findByEmailDomain(String domain)`：查询未删除、启用且邮箱以指定域结尾的用户。要求使用 `#{}` ，并为它增加集成测试。

[返回课件目录](#) · [下一节：MyBatis-Plus 简单实现原理](#)

第 9 节：MyBatis-Plus 简单实现原理

作者：xb 日期：2026-08-12

[上一节 · 返回目录](#)

这一课不要求背源码类名，目标是建立正确心智模型：MyBatis-Plus 没有绕过 MyBatis 和 JDBC，它主要在启动期注册通用 SQL、在运行期构造参数并通过插件改写 SQL。

1. 为什么 Mapper 接口没有实现类也能运行

启动时 `@MapperScan` 扫描 `UserMapper`，MyBatis 为接口创建 JDK 动态代理。Spring 容器中注入的 `UserMapper` 实际是代理对象。

调用：

```
userMapper.selectById(1L);
```

不是执行某个手写 Java 实现，而是代理根据“接口名 + 方法名”找到一个 `MappedStatement`，再交给 MyBatis Executor 执行。

2. MappedStatement 是什么

可以把它理解为一条已注册 SQL 的完整说明书，包含：

- 唯一 id；
- SQL 来源和动态 SQL 节点；
- 参数映射；
- 结果映射；
- SQL 类型（SELECT/INSERT/UPDATE/DELETE）；
- 缓存等配置。

XML 中 `UserMapper.findActiveAdults` 会在解析 XML 时注册。BaseMapper 的通用方法则由 MyBatis-Plus 在启动期注册。

3. “SQL 注入器”不是安全漏洞

MyBatis-Plus 源码中的 SQL Injector 指“向 MyBatis Configuration 注入通用 MappedStatement”。它和攻击领域的 SQL Injection 不是一回事。

概念流程：

```
发现 UserMapper extends BaseMapper<User>
→ 从 User 泛型读取 @TableName、@TableId、字段信息
→ 为 selectById/insert/updateById 等方法构建 SQL 模板
→ 注册为 MappedStatement
→ 运行时代理可按方法名找到并执行
```

因此，BaseMapper 能工作依赖实体元数据正确；表名、主键或字段映射错误会直接反映到生成 SQL。

4. Wrapper 如何变成 WHERE

`LambdaQueryWrapper` 内部记录条件片段、参数名和值。`User::getAge` 会通过 Lambda 元信息解析为实体属性，再通过表元数据得到数据库列 `age`。

最终 SQL 仍使用参数占位符，不是把普通值直接拼进去。Wrapper 是 SQL 构造器，不是业务规则层，也不会判断你的条件是否符合需求。

5. 执行器与 JDBC

简化后的运行链：

```
MapperProxy
→ SqlSessionTemplate (接入 Spring 事务)
→ Executor
→ StatementHandler
→ ParameterHandler 把 Java 值绑定到 ?
→ JDBC PreparedStatement
→ MySQL 执行
→ ResultSetHandler 映射为 User
```

TypeHandler 负责 Java 类型与 JDBC 类型转换，例如 `LocalDateTime` 与 `TIMESTAMP/DATETIME`。

6. 插件如何改写 SQL

`MybatisPlusInterceptor` 接入 MyBatis 插件机制，内部插件依次处理将要执行的 SQL：

- 乐观锁增加 version 更新与条件；
- 防攻击插件分析 UPDATE/DELETE 是否缺少条件；
- 分页插件解析 SELECT，生成 COUNT，并添加 LIMIT。

这解释了为何升级到 MyBatis-Plus 3.5.9+ 后分页要显式引入 SQL 解析器依赖：[pom.xml](#)。

7. Spring 事务如何接上 MyBatis

Spring 通过代理拦截 `@Transactional` 方法，把同一线程内的数据访问绑定到同一个事务资源。

`SqlSessionTemplate` 负责安全地获取/释放 `SqlSession`，并参与 Spring 事务同步。

这也是事务常要求从 Spring Bean 外部调用 public 方法的原因：只有经过代理，拦截器才有机会开启事务。

8. MyBatis-Plus 做了什么、没做什么

它做了：减少单表重复 CRUD、提供类型相对安全的条件构造、常用插件和生成工具。

它没做：设计数据模型、替你写业务规则、自动保证索引合理、消除 SQL 性能问题、自动让手写 SQL 遵守逻辑删除和权限规则。

成熟开发者仍需阅读最终 SQL、执行计划和数据库约束。

9. 自我讲解练习

请不用课件，向同伴讲清楚：

1. `UserMapper` 为什么可以注入？
2. `selectById` 的 SQL 在哪里来的？

3. Wrapper 如何避免硬编码列名?
4. 分页为何通常执行两条 SQL?
5. `@Transactional` 为什么可能因内部调用失效?

[返回课件目录](#) · [下一节：代码生成器、测试与排错](#)

第 10 节：代码生成器、测试与排错

作者：xb 日期：2026-08-12

[上一节](#) · [返回目录](#)

1. 代码生成器的定位

生成器根据数据库元数据创建 Entity、Mapper、Service、Controller 和 XML 骨架。它减少机械劳动，但不能生成正确业务逻辑，也不能替代代码审查。

查看完整配置：[MybatisPlusCodeGenerator](#)。

主要配置分组：

- DataSource：从哪个数据库读取表结构。
- GlobalConfig：作者、输出目录等。
- PackageConfig：生成到哪些包。
- StrategyConfig：包含哪些表、去掉什么前缀、覆盖策略。
- TemplateEngine：如何渲染模板，本项目用 Freemarker。

2. 运行生成器

先确保 MySQL 中已有 `demo_user`，再执行：

```
.\mvnw.cmd exec:java "-Dexec.mainClass=generator.com.xb.mybatisplus.MybatisPlusCodeGenerator"
```

结果位于 `target/generated-code`，不会覆盖手写教学源码。先检查生成结果，再按项目规范复制和修改。不要把生成目录直接当作完成的业务模块。

可用 JVM 参数临时覆盖数据源：

```
.\mvnw.cmd exec:java `
  "-Dexec.mainClass=generator.com.xb.mybatisplus.MybatisPlusCodeGenerator" `
  "-Ddb.url=jdbc:mysql://localhost:3306/mybatis_plus_demo" `
  "-Ddb.username=root" `
  "-Ddb.password=你的密码"
```

3. 集成测试为什么使用 H2

`UserMapperIntegrationTest` 验证 Mapper、XML、逻辑删除和分页，但使用内存 H2，避免测试误改共享 MySQL。

```
.\mvnw.cmd clean test
```

当前测试验证：

- 普通查询自动过滤逻辑删除记录；
- `deleteById` 实际更新 `deleted`；
- XML 参数与结果映射正确；
- 分页记录数与 `total` 正确；
- 新增时自增主键、业务默认值和时间字段自动填充；
- 乐观锁正常更新 `version`，并拒绝旧 `version` 覆盖；

- required 查询把不存在记录转换成明确的资源不存在异常。

H2 的 MySQL 兼容模式并不等同真正 MySQL。使用数据库特有函数、索引和执行计划时，还需要隔离的 MySQL 集成环境。

4. 推荐调试顺序

出现数据库问题时，从外向内检查：

1. HTTP 状态码和响应体是什么？
2. Controller 是否收到预期参数？
3. 控制台 `Preparing SQL` 是否正确？
4. `Parameters` 的值和类型是否正确？
5. 把等价 SQL 放进数据库客户端能否执行？
6. 表结构、字段、数据和账号权限是否符合预期？
7. 异常最底层的 `Caused by` 是什么？

不要只看最上面一句“Bean 创建失败”；根因通常在异常链底部。

5. 常见错误速查

现象	常见原因	检查点
Mapper 无法注入	未扫描、包路径错误	<code>@MapperScan</code>
Table doesn't exist	库名/表名/Profile 错误	JDBC URL、 <code>@TableName</code>
Unknown column	实体与表不一致	字段、 <code>@TableField</code> 、迁移脚本
Invalid bound statement	XML 未加载或 namespace/id 错	mapper-locations、接口方法
参数 not found	缺少或写错 <code>@Param</code>	Mapper 与 XML 参数名
分页没效果	未配置分页插件/解析器	<code>MybatisPlusConfig</code> 、pom
乐观锁一直失败	version 不是最新值	先查询再更新
被删除数据仍出现	手写 SQL 忘记过滤	<code>deleted=0</code>
Communications link failure	地址、端口、VPN、网络	DNS、端口、内网路由

6. 最终构建检查

```

.\mvnw.cmd clean test
.\mvnw.cmd -DskipTests package
java -jar target/mybatis-plus-demo-1.0.0.jar --spring.profiles.active=local

```

构建成功只能证明编译和自动测试通过；还应调用关键接口做冒烟测试。

[返回课件目录](#)